Note de séance SQL n°10

Objets programmables : procédures, fonctions, déclencheurs, événements | 2026-10-29

1. Les notions

Objet	Règle à retenir
Variables	@nom : variable de session (SET @x = 1 ; SELECT ... INTO @x). DECLARE : variable locale, en tête de bloc,
DELIMITER	Le client coupe une instruction au premier ; : on change le délimiteur ($$) pour envoyer le CREATE entier, puis
Procédure	CALL nom(...). Paramètres IN et OUT. IF / ELSEIF / ELSE, WHILE. Pas de CREATE OR REPLACE dans MyS
Fonction	RETURNS une valeur, utilisable dans un SELECT. Déclaration obligatoire : DETERMINISTIC (résultat selon les
NULL et agrégats	SUM sans ligne renvoie NULL, et SELECT ... INTO remplace le DEFAULT : toujours COALESCE(SUM(x), 0).
Erreurs	SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...' : erreur 1644. DECLARE EXIT HANDLER FOR SQL
Déclencheur	BEFORE / AFTER, INSERT / UPDATE / DELETE, FOR EACH ROW. NEW (valeurs futures) et OLD (valeurs p
Événement	Tâche planifiée : ON SCHEDULE EVERY ... / AT ..., STARTS, ENDS. Nécessite event_scheduler = ON.
Bonnes pratiques	Garder les scripts CREATE (avec DROP IF EXISTS) dans Git. Préférer une contrainte CHECK à un déclenche

2. Préparation

```
USE formation;
DROP TABLE IF EXISTS commandes_lab;
DROP TABLE IF EXISTS livraisons_lab;
DROP TABLE IF EXISTS livraisons_archive;
CREATE TABLE commandes_lab LIKE commandes;
INSERT INTO commandes_lab SELECT * FROM commandes;
CREATE TABLE livraisons_lab LIKE livraisons;
INSERT INTO livraisons_lab SELECT * FROM livraisons;
CREATE TABLE livraisons_archive LIKE livraisons;
```

3. Partie 1 : variables et procédures

1 : Variables

```
SET @seuil = 100;
SELECT id, produit, montant FROM commandes_lab WHERE montant > @seuil;
SELECT COUNT(*) INTO @nb FROM commandes_lab;
SELECT @nb;
```

Telephone, Ordinateur, Tablette, Montre, puis 7.

2 : Procédure sans paramètre

```
DROP PROCEDURE IF EXISTS p_resume_commandes;
DELIMITER $$
CREATE PROCEDURE p_resume_commandes()
BEGIN
    SELECT COUNT(*) AS nb_commandes, SUM(montant) AS ca,
           ROUND(AVG(montant), 2) AS panier_moyen
    FROM commandes_lab;
END$$
DELIMITER ;
CALL p_resume_commandes();
```

7, 1555, 222,14.

3-4 : Paramètres IN et OUT

```
DROP PROCEDURE IF EXISTS p_commandes_client;
```

```
DELIMITER $$
CREATE PROCEDURE p_commandes_client(IN p_client_id INT)
BEGIN
    SELECT id, produit, montant FROM commandes_lab
    WHERE client_id = p_client_id ORDER BY id;
END$$
DELIMITER ;

DROP PROCEDURE IF EXISTS p_total_client;
DELIMITER $$
CREATE PROCEDURE p_total_client(IN p_client_id INT, OUT p_total INT)
BEGIN
    SELECT COALESCE(SUM(montant), 0) INTO p_total
    FROM commandes_lab WHERE client_id = p_client_id;
END$$
DELIMITER ;
CALL p_total_client(4, @t);
SELECT @t;
```

Client 1 : 290 ; client 2 : 800 ; client 4 : 0 (grâce à COALESCE).

5 : IF et variables locales

```
DROP PROCEDURE IF EXISTS p_niveau_client;
DELIMITER $$
CREATE PROCEDURE p_niveau_client(IN p_client_id INT)
BEGIN
    DECLARE v_total INT DEFAULT 0;
    DECLARE v_niveau VARCHAR(10);
    SELECT COALESCE(SUM(montant), 0) INTO v_total
    FROM commandes_lab WHERE client_id = p_client_id;
    IF v_total >= 500 THEN SET v_niveau = 'Or';
    ELSEIF v_total >= 100 THEN SET v_niveau = 'Argent';
    ELSE SET v_niveau = 'Bronze';
    END IF;
    SELECT p_client_id AS client_id, v_total AS total, v_niveau AS niveau;
END$$
DELIMITER ;
```

Clients 1 à 5 : Argent, Or, Bronze, Bronze (total 0), Argent.

6 : Procédure qui modifie, testée en transaction

```
DROP PROCEDURE IF EXISTS p_appliquer_remise;
DELIMITER $$
CREATE PROCEDURE p_appliquer_remise(IN p_seuil INT, IN p_pct INT)
BEGIN
    UPDATE commandes_lab
    SET montant = ROUND(montant * (100 - p_pct) / 100)
    WHERE montant >= p_seuil;
    SELECT ROW_COUNT() AS lignes_modifiees;
END$$
DELIMITER ;
START TRANSACTION;
CALL p_appliquer_remise(300, 10);
SELECT id, produit, montant FROM commandes_lab WHERE montant >= 270;
ROLLBACK;
SELECT id, produit, montant FROM commandes_lab WHERE id IN (3, 6);
```

2 lignes modifiées : Ordinateur 720, Tablette 270 ; après ROLLBACK : 800 et 300.

7-8 : Gérer les procédures et boucle WHILE

```
SHOW PROCEDURE STATUS WHERE Db = 'formation';
SHOW CREATE PROCEDURE p_niveau_client;
-- modifier = DROP puis CREATE (seuil Or à 1000 : le client 2 passe à Argent)
DROP PROCEDURE IF EXISTS p_generer_commandes_test;
```

```
DELIMITER $$
CREATE PROCEDURE p_generer_commandes_test(IN p_n INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_id INT;
    SELECT COALESCE(MAX(id), 0) INTO v_id FROM commandes_lab;
    WHILE i <= p_n DO
        SET v_id = v_id + 1;
        INSERT INTO commandes_lab (id, client_id, produit, montant,
                                   date_commande, produit_id)
        VALUES (v_id, 1, 'Test', 10 * i, CURDATE(), 1);
        SET i = i + 1;
    END WHILE;
END$$
DELIMITER ;
CALL p_generer_commandes_test(5);
```

12 commandes, id maximum 12.

4. Partie 2 : fonctions, SIGNAL, HANDLER

1-3 : Fonctions stockées

```
DROP FUNCTION IF EXISTS f_niveau;
DELIMITER $$
CREATE FUNCTION f_niveau(p_total INT) RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    DECLARE v_niveau VARCHAR(10);
    IF p_total >= 500 THEN SET v_niveau = 'Or';
    ELSEIF p_total >= 100 THEN SET v_niveau = 'Argent';
    ELSE SET v_niveau = 'Bronze';
    END IF;
    RETURN v_niveau;
END$$
DELIMITER ;

DROP FUNCTION IF EXISTS f_total_client;
DELIMITER $$
CREATE FUNCTION f_total_client(p_client_id INT) RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_total INT;
    SELECT COALESCE(SUM(montant), 0) INTO v_total
    FROM commandes_lab WHERE client_id = p_client_id;
    RETURN v_total;
END$$
DELIMITER ;

SELECT c.prenom, COALESCE(SUM(o.montant), 0) AS total,
       f_niveau(COALESCE(SUM(o.montant), 0)) AS niveau
FROM clients c LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY c.id, c.prenom;
SELECT prenom, f_niveau(f_total_client(id)) AS niveau FROM clients;
```

Awa 290 Argent, Jean 800 Or, Marie 45 Bronze, Patrick 0 Bronze, Grace 300 Argent, David, Sarah, Eric 0 Bronze. La fonction lit commandes_lab (et non une autre copie).

4 : Erreur 1418

Avec le journal binaire, MySQL doit savoir qu'une fonction donnera le même résultat si elle est rejouée sur un autre serveur : DETERMINISTIC (dépend des seuls paramètres) ou READS SQL DATA (lit des tables).

5 : SIGNAL

```
DROP PROCEDURE IF EXISTS p_ajouter_livraison;
```

```
DELIMITER $$
CREATE PROCEDURE p_ajouter_livraison(IN p_commande_id INT, IN p_cout DECIMAL(8,2))
BEGIN
    IF NOT EXISTS (SELECT 1 FROM commandes_lab WHERE id = p_commande_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Commande inexistante';
    END IF;
    IF p_cout < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Le coût ne peut pas être négatif';
    END IF;
    INSERT INTO livraisons_lab (commande_id, cout) VALUES (p_commande_id, p_cout);
END$$
DELIMITER ;
```

CALL (2, 20.00) : ajoute (4 lignes, statut en préparation) ; (99, 10.00) et (2, -5.00) : erreur 1644 avec le message.

6 : HANDLER et transaction (tout ou rien)

```
DROP PROCEDURE IF EXISTS p_archiver_livraison;
DELIMITER $$
CREATE PROCEDURE p_archiver_livraison(IN p_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Erreur : opération annulée' AS message;
    END;
    START TRANSACTION;
    INSERT INTO livraisons_archive SELECT * FROM livraisons_lab WHERE id = p_id;
    DELETE FROM livraisons_lab WHERE id = p_id;
    COMMIT;
    SELECT 'Livraison archivée' AS message;
END$$
DELIMITER ;
```

CALL (1) : 3 lignes dans livraisons_lab, 1 dans l'archive. Après un INSERT manuel de la livraison 2 dans l'archive, CALL (2) : clé en double, ROLLBACK, la livraison 2 reste dans livraisons_lab.

5. Partie 3 : déclencheurs et événements

Préparation : Journal et historique

```
DROP TABLE IF EXISTS commandes_lab;
DROP TABLE IF EXISTS journal_commandes_lab;
DROP TABLE IF EXISTS historique_total;
CREATE TABLE commandes_lab LIKE commandes;
INSERT INTO commandes_lab SELECT * FROM commandes;
CREATE TABLE journal_commandes_lab (
    id INT AUTO_INCREMENT PRIMARY KEY,
    action VARCHAR(10) NOT NULL,
    commande_id INT NOT NULL,
    ancien_montant INT,
    nouveau_montant INT,
    date_action DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE historique_total (
    id INT AUTO_INCREMENT PRIMARY KEY,
    date_mesure DATETIME NOT NULL,
    nb_commandes INT NOT NULL,
    ca INT NOT NULL
);
```

2-4 : Déclencheurs d'audit

```
DROP TRIGGER IF EXISTS trg_commandes_lab_ai;
DELIMITER $$
CREATE TRIGGER trg_commandes_lab_ai AFTER INSERT ON commandes_lab
```

```
FOR EACH ROW
BEGIN
    INSERT INTO journal_commandes_lab (action, commande_id, ancien_montant, nouveau_montant)
    VALUES ('INSERT', NEW.id, NULL, NEW.montant);
END$$
DELIMITER ;

DROP TRIGGER IF EXISTS trg_commandes_lab_au;
DELIMITER $$
CREATE TRIGGER trg_commandes_lab_au AFTER UPDATE ON commandes_lab
FOR EACH ROW
BEGIN
    IF OLD.montant <> NEW.montant THEN
        INSERT INTO journal_commandes_lab (action, commande_id, ancien_montant, nouveau_montant)
        VALUES ('UPDATE', NEW.id, OLD.montant, NEW.montant);
    END IF;
END$$
DELIMITER ;

DROP TRIGGER IF EXISTS trg_commandes_lab_ad;
DELIMITER $$
CREATE TRIGGER trg_commandes_lab_ad AFTER DELETE ON commandes_lab
FOR EACH ROW
BEGIN
    INSERT INTO journal_commandes_lab (action, commande_id, ancien_montant, nouveau_montant)
    VALUES ('DELETE', OLD.id, OLD.montant, NULL);
END$$
DELIMITER ;
```

Insertion (8, 180), mise à jour du montant à 200 (la modification du seul produit n'est pas journalisée : le montant est inchangé), suppression : INSERT 8 NULL 180 ; UPDATE 8 180 200 ; DELETE 8 200 NULL.

5 : BEFORE INSERT : valider et corriger

```
DROP TRIGGER IF EXISTS trg_commandes_lab_bi;
DELIMITER $$
CREATE TRIGGER trg_commandes_lab_bi BEFORE INSERT ON commandes_lab
FOR EACH ROW
BEGIN
    IF NEW.montant IS NULL OR NEW.montant <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Montant invalide';
    END IF;
    SET NEW.produit = TRIM(NEW.produit);
    IF NEW.date_commande IS NULL THEN
        SET NEW.date_commande = CURDATE();
    END IF;
END$$
DELIMITER ;
```

Produit sans espaces, date du jour ; montant -5 : erreur 1644, aucune ligne ajoutée ni journalisée.

6 : Gérer les déclencheurs

```
SHOW TRIGGERS LIKE 'commandes_lab';
DROP TRIGGER trg_commandes_lab_bi;
```

Pour une règle simple, une contrainte CHECK (montant > 0) est préférable : elle fait partie de la table, se voit dans SHOW CREATE TABLE, couvre aussi les UPDATE et ne peut pas être supprimée par oubli comme un déclencheur.

7 : Événement planifié

```
SHOW VARIABLES LIKE 'event_scheduler';
SET GLOBAL event_scheduler = ON;
DROP EVENT IF EXISTS ev_snapshot_total;
CREATE EVENT ev_snapshot_total
ON SCHEDULE EVERY 1 MINUTE
STARTS CURRENT_TIMESTAMP
```

```
ENDS CURRENT_TIMESTAMP + INTERVAL 4 MINUTE
DO
    INSERT INTO historique_total (date_mesure, nb_commandes, ca)
    SELECT NOW(), COUNT(*), COALESCE(SUM(montant), 0) FROM commandes_lab;
-- attendre environ 3 minutes, puis :
SELECT * FROM historique_total;
SHOW EVENTS;
DROP EVENT IF EXISTS ev_snapshot_total;
```

Au moins 3 lignes. Le DROP EVENT ne vient qu'après l'attente, sinon l'événement est supprimé avant d'avoir produit quoi que ce soit.

8 : Réflexion

Déclencheur : audit complet et impossible à oublier côté application ; mais logique invisible, ralentissement des écritures, difficulté de débogage. Déclencheur : réagit aux modifications de données ; événement : réagit à l'horloge. CHECK pour une règle simple sur une valeur.

6. Pièges rencontrés

• **Syntaxe** : DROP ... IF EXISTS termine par ; ; END$$ sans point-virgule ; DELIMITER ; (et non DELIMITER $$;), sinon les instructions suivantes ne partent plus.

• **NULL** : SUM sans ligne donne NULL ; SELECT ... INTO remplace le DEFAULT ; utiliser COALESCE.

• **Nom de table en dur** : une fonction qui lit commandes_bac au lieu de commandes_lab donne les mêmes chiffres par hasard, puis échoue si la copie disparaît (MySQL ne vérifie la table qu'à l'appel).

• **Tout ou rien** : plusieurs étapes dans une transaction avec un HANDLER ; sinon une erreur laisse une ligne en double ou perdue.

• **Fonction ligne par ligne** : un million de lignes, un million d'appels ; un JOIN avec GROUP BY fait le même travail en un seul passage.

• **Déclencheur invisible** : un BEFORE INSERT ne contrôle pas les UPDATE ; préférer CHECK pour une règle simple.

• **Événement** : activer le planificateur, attendre, puis supprimer l'événement, dans cet ordre.

• **Versionner** : le code vit dans la base ; garder les scripts CREATE dans le dépôt Git.